CS 3563: Introduction to DBMS II

Homework 2: Storage Management and Indexing

Due Date and Time: 15 Feb. 2026, 11:59 PM IST

Instruction *Total Marks: 30*

1. The homework must be completed individually.
2. Referring to external resources beyond the class notes and reference textbooks is not permitted.
3. The submission must be a single PDF and submitted via Moodle.
4. Late submissions will be handled in accordance with the course policy.

Question 1 **(3 + 3 Marks)**

Standard buffer managers often assume that all blocks are of equal size and incur the same cost when read. Consider instead a buffer manager that:

- tracks the rate of reference of objects over a sliding window of the last 10 seconds,
- stores objects of varying sizes, and
- incurs different read costs for different objects.

The buffer manager assigns each object an eviction score that captures the expected cost per unit buffer space incurred by evicting that object. Formally, for an object i:

$$\text{EvictionScore}(i) = \frac{\text{AccessRate}(i) \times \text{ReadCost}(i)}{\text{Size}(i)}$$

Objects with lower eviction scores are preferred eviction candidates.

Assume that total buffer capacity is 10 MB. Further, at a certain point in time, the buffer pool contains the following objects:

Object	Size (MB)	Read Cost (ms)	References in last 10 seconds
A	4	50	20
B	2	200	5
C	3	30	2
D	1	100	8

1. A new object E of size 3 MB needs to be brought into the buffer pool. Using the eviction score defined above, determine which object(s) should be evicted to make space for object E.
2. Suppose the buffer manager is allowed to evict multiple objects if needed. Is it preferable to evict (a) object A, or (b) objects C and D together?

Justify your answer numerically using the eviction scores.

Question 2 **(3 + 3 Marks)**

A relation R is stored as a heap file consisting of fixed-size disk blocks. Each block can store exactly 100 records. The system maintains a free-space map (FSM) using 2 bits per block. The meaning of the FSM bits is as follows: If the block is between 0 and 30 percent full the bits are 00, between 30 and 60 percent the bits are 01, between 60 and 90 percent the bits are 10, and above 90 percent the bits are 11.

At time T_0, the FSM is correct and reflects the actual block occupancies. The number of records in blocks 1–6 at time T_0 is given below.

Block	1	2	3	4	5	6
Records at T_0	62	85	20	45	75	95

Between time T_0 and T_1, the following updates occur. During this interval, the FSM is not updated.

Block	1	2	3	4	5
During (T_0, T_1)	+5 inserts	+10 inserts	+60 inserts	−10 deletes	+20 inserts

1. Determine which blocks have stale (outdated) FSM entries at time T_1.
2. At time T_1, a transaction attempts to insert 30 new records. The system uses the following insertion policy:
   - FSM entries are considered once, in increasing block number order.
   - When an FSM entry is considered, the corresponding block is deemed able to *fully accommodate* the batch of records if, according to the FSM, it can guarantee sufficient free space.
   - If such a block is found, the system attempts to insert the records into that block; insertion stops if the block becomes full in reality.
   - If there are any remaining records, the traversal continues with the same logic for these remaining records over the remaining FSM entries.
   - After the FSM traversal completes, if any records remain, a new block is allocated, and the remaining records are inserted into it.

   After the insertion completes, the system performs a *full* FSM refresh, recomputing the FSM from the actual block occupancies. What is the final FSM state for blocks 1–6 after the refresh?

Question 3 **(6 Marks)**

A B^+ tree is constructed from the following set of insert and delete operations (I = Insert, D = Delete):

$$I\,2, I\,3, I\,5, I\,7, I\,11, I\,17, I\,19, I\,23, I\,29, I\,31, I\,9, I\,10, I\,8, D\,23, D\,19, D\,29, D\,11$$

Starting from an empty database, show the step-by-step construction of the B^+ tree after each operation. Assume that the maximum number of pointers in a node is 4, and that the B^+ tree is constructed with left pointers corresponding to $<$ and right pointers corresponding to $\geq$.

Question 4 **(6 Marks)**

For the same set of operations as above, show the step-by-step construction of the extendible hashing index structure. Assume you start with an empty database, the hash function is $h(x) = x \ mod \ 8$, and buckets can hold up to 3 records. Also, assume that the most significant bits in the hash value are used for identifying buckets.

Question 5 **(6 Marks)**

For the same set of operations as above, show the step-by-step construction of the linear hashing index structure. Assume you start with two index blocks, and that each bucket can hold up to 3 records. The hash function initially is $h(x) = x \ mod \ 2$, and progressively evolves by revealing one additional least significant bit (i.e., the effective modulo doubles) for buckets that undergo splitting. The buckets are split in a round-robin fashion using a split pointer.

A split is triggered whenever a new overflow block is assigned to the index, and exactly one split is performed per such event. Note that during insertion, the effective hash function used for a key depends on whether the target bucket has already been split. Finally, assume that no bucket merging is performed during deletions.